# HOSPITAL & CLINIC MANAGEMENT DATABASE SYSTEM

Full Project Report & Documentation

# TABLE OF CONTENTS

SECTION 1 — CONCEPTUAL DESIGN ..... 3
1.1 Problem Description and Requirements Analysis ..... 3
1.2 Main Data Objects and Business Rules ..... 4
1.3 Use-Case Diagram ..... 6
1.4 Activity Diagram — Appointment Booking Process ..... 7
1.5 Sequence Diagram — Appointment Request Flow ..... 8
SECTION 2 — LOGICAL DESIGN ..... 9
2.1 Entity-Relationship Diagram ..... 9
2.2 Entities, Attributes, and Relationships ..... 10
2.3 Relationship Cardinalities and Constraints ..... 14
2.4 Relational Schema Transformation ..... 15
2.5 Normalization Process (1NF, 2NF, 3NF) ..... 16
2.6 Relational Algebra Operations — All 30 Queries ..... 17
SECTION 3 — PHYSICAL IMPLEMENTATION ..... 19
3.1 DDL — Database and Table Creation ..... 19
3.2 DML — Data Insertion (42 rows per table) ..... 21
3.3 DQL — Select Queries (30 Queries) ..... 21
3.4 Views — 8 Reusable Query Objects ..... 24
3.5 Indexes — 15 Performance Optimizations ..... 26
3.6 Triggers — 7 Business Rule Automations ..... 28
3.7 Stored Procedures — 8 Reusable Operations ..... 30
3.8 DCL — Role-Based Access Control ..... 32
3.9 Database Deployment ..... 32
SECTION 4 — REPORT AND DOCUMENTATION ..... 33
4.1 Project Conclusions ..... 34
4.2 Key Technical Decisions and Justifications ..... 35
4.3 Requirements Coverage Summary ..... 36

# SECTION 1 — CONCEPTUAL DESIGN

## 1.1 Problem Description and Requirements Analysis

Hospital and clinic management systems coordinate the complete lifecycle of patient care — from registration and appointment scheduling through doctor assignment, billing, and post-visit feedback — reliably and with full data integrity.

The central operational challenge is coordinating medical staff availability (doctors and rooms) with patient demand (appointment requests) while ensuring every state change is valid, every financial record is accurate, and all clinical business rules are enforced automatically even if the application layer is bypassed.

### Functional Requirements

- Patients register with a unique email address and book appointments by choosing a doctor and department.
- Doctors register with a unique licence number and manage their availability status (Available / Busy / Off-Duty).
- The system assigns exactly one doctor and one department to each appointment.
- Appointments record the reason for visit, scheduled time, end time, fee, and auto-calculated duration.
- Bills are recorded per appointment, capturing payment method (Cash, Card, Insurance) and outcome (Paid / Pending / Failed).
- Patients and doctors can rate each other on a 1–5 scale after a completed appointment.
- Promotional discount codes can be applied to reduce consultation fees.
- Administrators manage patients, doctors, database access, and generate operational and financial reports.

### Non-Functional Requirements

| Category | Requirement | How Implemented |
|---|---|---|
| Integrity | All constraints enforced at DB engine level | 10 FKs, 12 CHECKs, 6 UNIQUEs, 8 DEFAULTs across all tables |
| Performance | Fast queries on high-traffic columns | 15 non-clustered indexes on FK, status, date and fee columns |
| Security | Least-privilege access per user role | 3 DCL roles — app user (no DELETE), report (SELECT only), DBA (full) |
| Consistency | Derived data always accurate without manual updates | 7 triggers maintain Duration, Rating, doctor Status automatically |
| Auditability | Rules enforced even if application is bypassed | INSTEAD OF triggers block invalid state transitions before storage |
| Scalability | Schema handles millions of rows without changes | Full 3NF normalisation; all relationships via FK references |

## 1.2 Main Data Objects and Business Rules

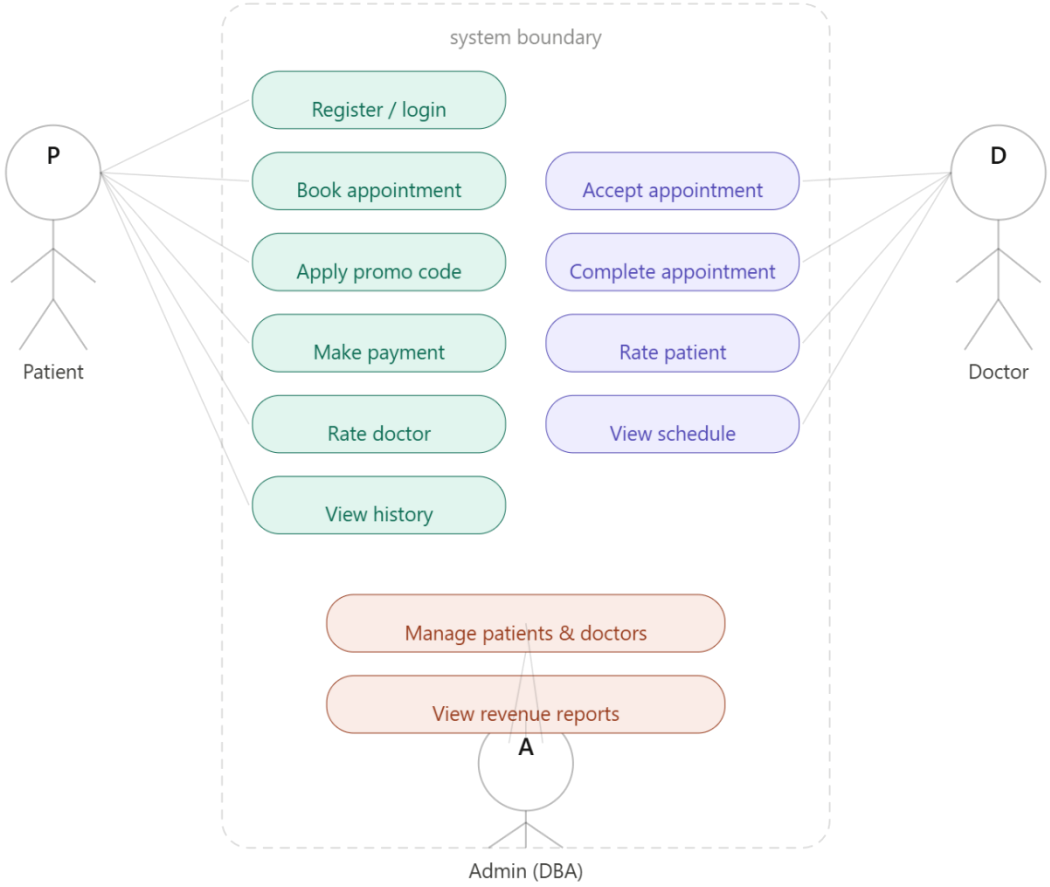
| Data Object | Role in System | Key Business Rules |
|---|---|---|
| patients | Registered patient who books appointments | Unique email; one active Scheduled appointment at a time (trigger BR-1) |
| doctors | Registered doctor who fulfils appointments | Unique licence; Rating auto-recalculated; Status auto-managed (BR-4, BR-6, BR-7) |
| departments | Hospital department hosting appointments | Referenced by appointments; enables department-level revenue analytics |
| rooms | Physical room within a department | Capacity 1–20; type enum (General/ICU/Operating/Recovery/Consultation) |
| appointments | Central fact — a single visit from patient to doctor | Links patient, doctor, department; status Scheduled/Completed/Cancelled |
| bills | Financial record for an appointment | One bill per appointment; cancelled appointments cannot be billed (trigger BR-3) |
| ratings | Post-appointment mutual feedback record | One rating per appointment (UNIQUE on AppointmentID); DoctorRating mandatory |
| promotions | Discount voucher applied to an appointment | Discount 0–100% (CHECK); expiry date enforced at query time |

### Business Rules Enforced at Database Level

| ID | Business Rule | Enforcement Mechanism |
|---|---|---|
| BR-1 | A patient can have only one active (Scheduled) appointment at a time | INSTEAD OF INSERT trigger — trg_no_concurrent_appointments |
| BR-2 | Completed appointments can never be deleted | AFTER DELETE + ROLLBACK — trg_prevent_delete_completed |
| BR-3 | Bill cannot be created for a Cancelled appointment | INSTEAD OF INSERT trigger — trg_validate_bill_appointment |
| BR-4 | Doctor average rating is always current after each new rating | AFTER INSERT trigger — trg_update_doctor_rating |
| BR-5 | Duration is auto-calculated when EndTime is set | AFTER UPDATE trigger — trg_calc_duration |
| BR-6 | Doctor status becomes Busy when a Scheduled appointment is inserted | AFTER INSERT trigger — trg_doctor_busy_on_appointment |
| BR-7 | Doctor status resets to Available when appointment ends or is cancelled | AFTER UPDATE trigger — trg_doctor_available_on_complete |
| BR-8 | Promo discount must be between 0% and 100% | CHECK constraint on promotions.Discount |
| BR-9 | Room capacity must be between 1 and 20 | CHECK constraint on rooms.Capacity |
| BR-10 | Bill payment method must be Cash, Card, or Insurance | CHECK constraint on bills.Method |
| BR-11 | Appointment status must be Scheduled, Completed, or Cancelled | CHECK constraint on appointments.Status |
| BR-12 | Doctor rating must be between 0.0 and 5.0 | CHECK constraint on doctors.Rating |

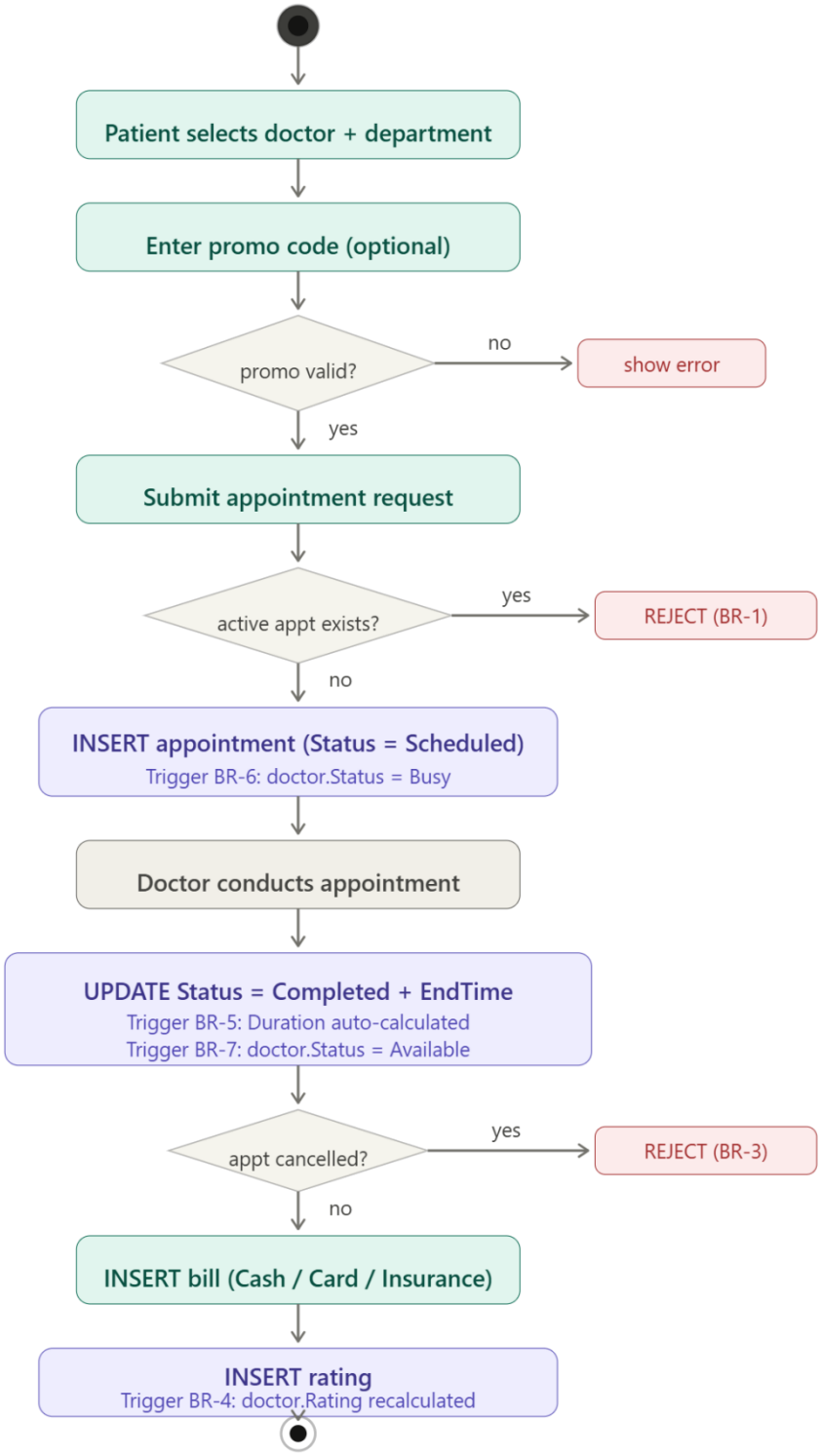
## 1.3 Use-Case Diagram

The use-case diagram shows the three actors — Patient, Doctor, and Admin — and what each one can do in the system. The key point is that database triggers enforce rules regardless of which actor triggers the action.



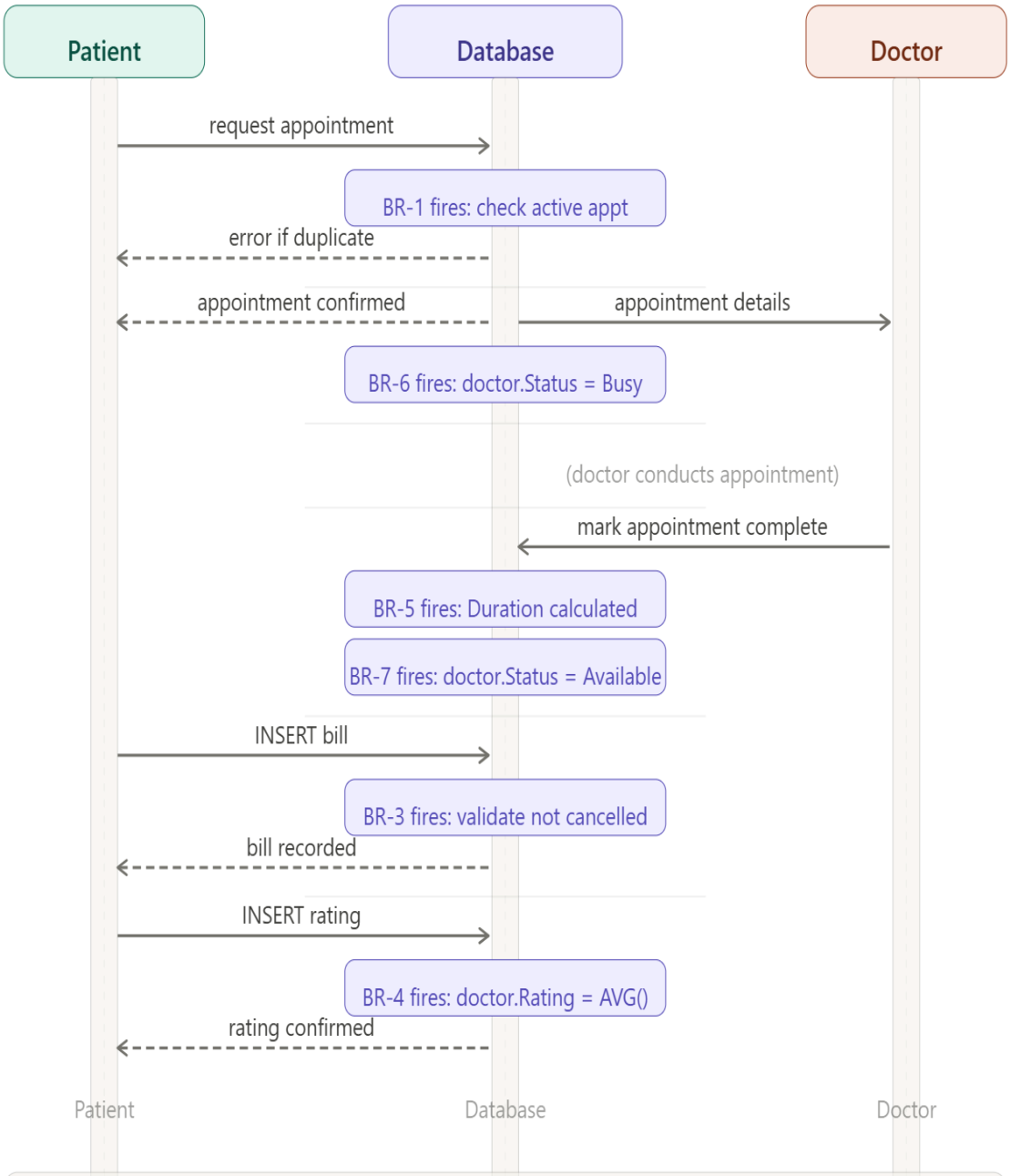
## 1.4 Activity Diagram — Appointment Booking Process

The activity diagram shows what happens step by step when a patient books an appointment — including every point where the database fires a trigger automatically.



## 1.5 Sequence Diagram — Appointment Request Flow

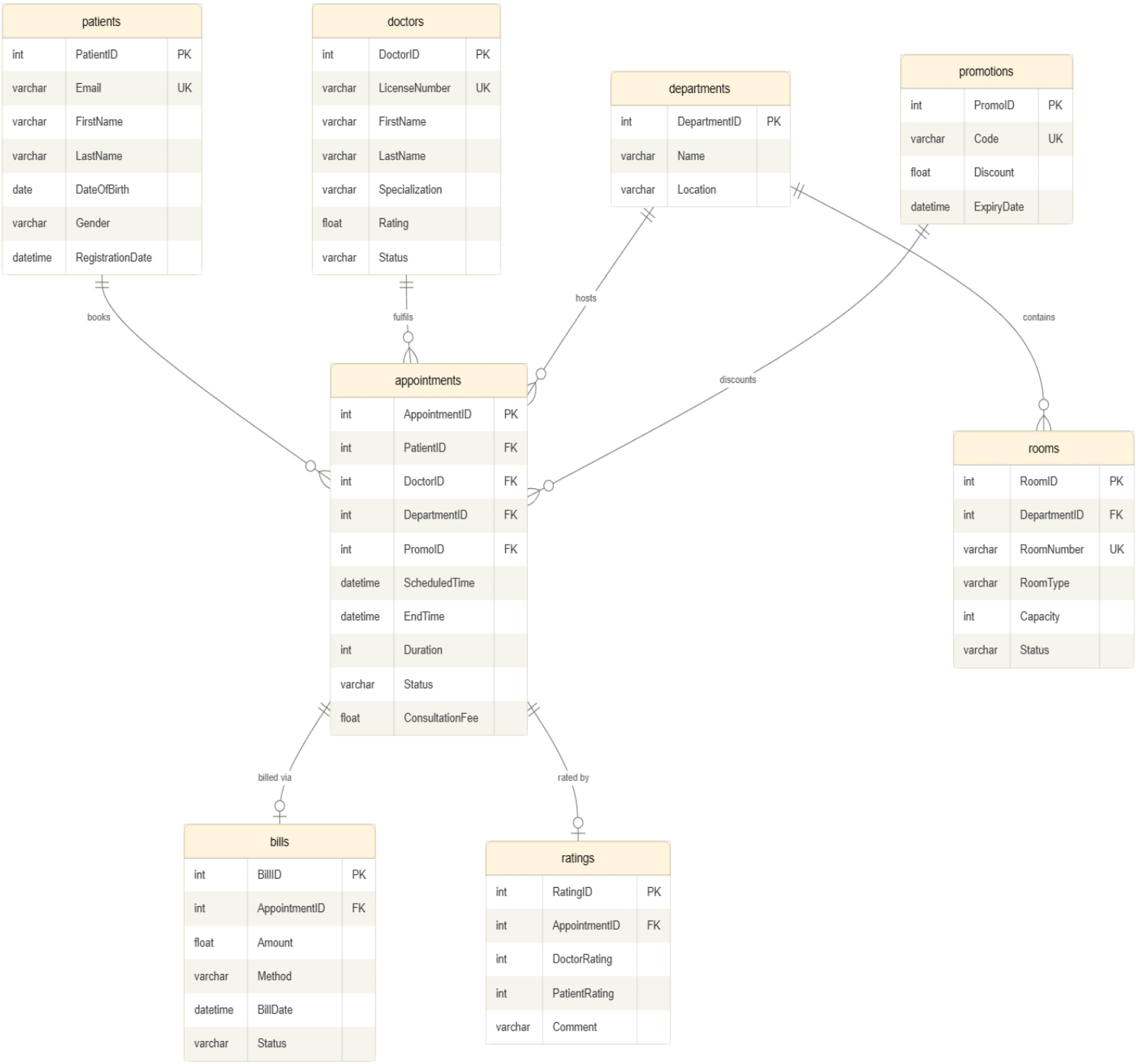
The sequence diagram shows the exact messages exchanged between the three participants — Patient, Database, and Doctor — in time order from top to bottom. This is the most technical of the four diagrams and is what examiners often ask you to explain.



# SECTION 2 — LOGICAL DESIGN

## 2.1 Entity-Relationship Diagram

The ER diagram shows all 8 tables and how they connect to each other.



## 2.2 Entities, Attributes, and Relationships

**Table: patients**

Registered patients who book appointments.

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| PatientID | INT | PK, IDENTITY(1,1) | Auto-generated unique row identifier |
| FirstName | VARCHAR(50) | NOT NULL | Patient first name |
| LastName | VARCHAR(50) | NOT NULL | Patient last name |
| Email | VARCHAR(100) | NOT NULL, UNIQUE, CHECK format | Login email — must match %@%.% pattern |
| Phone | VARCHAR(20) | NULL | Optional contact phone number |
| DateOfBirth | DATE | NOT NULL | Patient date of birth |
| Gender | VARCHAR(10) | NOT NULL, CHECK enum | Male / Female / Other |
| RegistrationDate | DATETIME | NOT NULL, DEFAULT GETDATE() | Timestamp when account was created |

**Table: doctors**

Registered doctors who fulfil appointment requests.

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| DoctorID | INT | PK, IDENTITY(1,1) | Auto-generated unique row identifier |
| FirstName | VARCHAR(50) | NOT NULL | Doctor first name |
| LastName | VARCHAR(50) | NOT NULL | Doctor last name |
| LicenseNumber | VARCHAR(30) | NOT NULL, UNIQUE | Medical licence — unique across doctor pool |
| Specialization | VARCHAR(100) | NOT NULL | Medical specialty (e.g. Cardiology) |
| Rating | FLOAT | NOT NULL, DEFAULT 5.0, CHECK 0-5 | Average patient rating; auto-recalculated by trigger |
| Status | VARCHAR(20) | NOT NULL, CHECK enum | Available / Busy / Off-Duty — managed by triggers BR-6 and BR-7 |

### Table: departments

Hospital departments that host appointments and contain rooms.

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| DepartmentID | INT | PK, IDENTITY(1,1) | Auto-generated unique row identifier |
| Name | VARCHAR(100) | NOT NULL | Department name (e.g. Cardiology) |
| Location | VARCHAR(100) | NOT NULL | Physical location (e.g. Building A, Floor 3) |

### Table: rooms

Physical rooms within departments. Capacity and type are validated by CHECK constraints.

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| RoomID | INT | PK, IDENTITY(1,1) | Auto-generated unique row identifier |
| DepartmentID | INT | NOT NULL, FK → departments | Department this room belongs to |
| RoomNumber | VARCHAR(20) | NOT NULL, UNIQUE | Unique room identifier (e.g. A301) |
| RoomType | VARCHAR(30) | NOT NULL, CHECK enum | General / ICU / Operating / Recovery / Consultation |
| Capacity | INT | NOT NULL, CHECK 1–20 | Maximum patient capacity — values outside 1–20 rejected |
| Status | VARCHAR(20) | NOT NULL, CHECK enum | Available / Occupied / Maintenance |

### Table: appointments

Central fact table — records every appointment in the hospital system.

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| AppointmentID | INT | PK, IDENTITY(1,1) | Auto-generated unique row identifier |
| PatientID | INT | NOT NULL, FK → patients CASCADE | Patient who booked the appointment |
| DoctorID | INT | NOT NULL, FK → doctors | Doctor assigned to this appointment |
| DepartmentID | INT | NOT NULL, FK → departments | Department where appointment takes place |
| ScheduledTime | DATETIME | NOT NULL | Date and time the appointment begins |
| EndTime | DATETIME | NULL | Completion time — NULL while appointment is in progress |
| Duration | INT | NULL — derived by trigger BR-5 | Minutes elapsed — auto-set by trigger when EndTime is written |
| ReasonForVisit | VARCHAR(300) | NOT NULL | Chief complaint or reason for the visit |
| Status | VARCHAR(20) | NOT NULL, CHECK enum | Scheduled / Completed / Cancelled |
| ConsultationFee | FLOAT | NOT NULL, DEFAULT 0, CHECK >= 0 | Total fee charged — cannot be negative |
| PromoID | INT | NULL, FK → promotions | Optional promo code — NULL when no discount applied |

### Table: bills

Financial transaction record for each appointment.

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| BillID | INT | PK, IDENTITY(1,1) | Auto-generated unique row identifier |
| AppointmentID | INT | NOT NULL, FK → appointments CASCADE | The appointment this bill covers |
| Amount | FLOAT | NOT NULL, CHECK >= 0 | Amount charged — cannot be negative |
| Method | VARCHAR(20) | NOT NULL, CHECK enum | Cash / Card / Insurance |
| BillDate | DATETIME | NOT NULL, DEFAULT GETDATE() | Timestamp when payment was processed |
| Status | VARCHAR(20) | NOT NULL, CHECK enum | Paid / Pending / Failed |

**Table: ratings**

Post-appointment feedback submitted by the patient (and optionally by the doctor).

| Column | Data Type | Constraints | Description |
|---|---|---|---|
| RatingID | INT | PK, IDENTITY(1,1) | Auto-generated unique row identifier |
| AppointmentID | INT | NOT NULL, FK → appointments CASCADE, UNIQUE | The appointment being rated — UNIQUE ensures at most one rating per appointment |
| DoctorRating | INT | NOT NULL, CHECK 1–5 | Patient's mandatory rating of the doctor |
| PatientRating | INT | NULL, CHECK NULL or 1–5 | Doctor's optional rating of the patient |
| Comment | VARCHAR(500) | NULL | Optional free-text feedback from the patient |

## 2.3 Relationship Cardinalities and Constraints

| Relationship | ERD Cardinality | SQL Cardinality | FK Constraint | Design Notes |
|---|---|---|---|---|
| patients → appointments | 0..1 to * | 1:N (patient optional on appt) | ON DELETE CASCADE | A patient may have zero appointments; one patient can have many over time |
| doctors → appointments | 0..1 to * | 1:N | No cascade | A doctor may have zero appointments; one doctor can fulfil many appointments |
| departments → appointments | 0..1 to * | 1:N | No cascade | One department can host many appointments across many doctors |
| departments → rooms | 0..1 to * | 1:N | No cascade | One department can contain many rooms of different types |
| appointments → bills | 0..1 to 1 | 1:0..1 (optional) | ON DELETE CASCADE | A scheduled appointment may not yet have a bill; max one bill per appointment |
| appointments → ratings | 0..1 to 1 | 1:0..1 (optional) | CASCADE + UNIQUE on AppointmentID | UNIQUE constraint on ratings.AppointmentID enforces one-rating-per-appointment |
| promotions → appointments | 0..1 to * | 1:N (optional FK) | No cascade, NULL | PromoID is NULL in appointments when no code was applied |

## 2.4 Relational Schema Transformation

The ER model maps to the relational schema following standard transformation rules. Each entity becomes a table; each attribute becomes a typed column. Single-column surrogate PKs (IDENTITY INT) are used throughout.

```
1 patients     ( PatientID PK, FirstName, LastName,
                 Email UQ+CHK, Phone NULL,
                 DateOfBirth, Gender CHK, RegistrationDate )

2 doctors      ( DoctorID PK, FirstName, LastName,
                 LicenseNumber UQ, Specialization,
                 Rating CHK, Status CHK )

3 departments  ( DepartmentID PK, Name, Location )

4 rooms        ( RoomID PK, DepartmentID FK→departments,
                 RoomNumber UQ, RoomType CHK, Capacity CHK, Status CHK )

5 promotions   ( PromoID PK, Code UQ, Discount CHK, ExpiryDate )

6 appointments ( AppointmentID PK,
                 PatientID    FK→patients    CASCADE,
                 DoctorID     FK→doctors,
                 DepartmentID FK→departments,
                 ScheduledTime, EndTime NULL,
                 Duration DERIVED, ReasonForVisit,
                 Status CHK, ConsultationFee CHK,
                 PromoID FK→promotions NULL )

7 bills        ( BillID PK,
                 AppointmentID FK→appointments CASCADE,
                 Amount CHK, Method CHK, BillDate, Status CHK )

8 ratings      ( RatingID PK,
                 AppointmentID FK+UQ→appointments CASCADE,
                 DoctorRating CHK, PatientRating NULL CHK, Comment NULL )
```

## 2.5 Normalization Process (1NF, 2NF, 3NF)

### First Normal Form — 1NF

Definition: Every column contains atomic (indivisible) values; all values in a column share the same data type; every row is uniquely identified by a primary key.

- Every column in every table holds a single, indivisible value — no arrays, CSV strings, or nested groups.
- Every table has a unique IDENTITY primary key guaranteeing distinct row identification.
- Example: Rather than storing "Cardiology, Building A" as a combined string in appointments, the system uses DepartmentID (INT FK) — a single atomic integer referencing a single atomic row in departments. Classic atomic decomposition.

### Second Normal Form — 2NF

Definition: In 1NF AND every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies).

- Every table uses a single-column surrogate PK (IDENTITY INT). With a single-column PK, partial functional dependency is structurally impossible.
- In appointments: ConsultationFee, Status, ScheduledTime all depend only on AppointmentID — no partial dependency exists.
- NOTE: Partial dependency requires a composite PK. Since all PKs here are single IDENTITY columns, 2NF is guaranteed by design.

### Third Normal Form — 3NF

Definition: In 2NF AND no non-key attribute is transitively dependent on the PK through another non-key attribute.

| Potential Transitive Dep. | Problem if Not Eliminated | Resolution Applied |
|---|---|---|
| Doctor name stored in appointments (DoctorName → DoctorID → name) | Updating a doctor's name would require updating every appointments row — data anomaly | appointments stores only DoctorID (FK). Doctor name retrieved via JOIN with doctors at query time. |
| Department name in appointments (DeptName → DepartmentID → name) | Renaming a department would corrupt all historical appointment records | appointments stores DepartmentID (FK only). Name retrieved via JOIN with departments. |
| Promo discount in appointments (DiscountPct → PromoID → discount) | If a promo code's discount changes, historical fees would appear incorrect | appointments stores PromoID (FK only). Discount retrieved via JOIN with promotions. |
| Duration from ScheduledTime+EndTime | Technically derivable — storing it could seem like a dependency on non-key attrs | Controlled performance denormalisation: stored for query efficiency; maintained by trigger BR-5. |

## 2.6 Relational Algebra Operations — All 30 Queries

### Selection σ — filter rows satisfying a condition (Q1–Q7)

| Q# | Op | Formal Expression | SQL Clause | Result |
|---|---|---|---|---|
| Q1 | σ | σ_{Status='Completed'} (appointments) | WHERE Status = 'Completed' | 40 completed appointment records |
| Q2 | σ | σ_{Status='Available'} (doctors) | WHERE Status = 'Available' | All currently available doctors |
| Q3 | σ | σ_{ConsultationFee > 150} (appointments) | WHERE Fee > 150 | Appointments with fee exceeding $150 |
| Q4 | σ | σ_{RegistrationDate > '2023-06-01'} (patients) | WHERE RegistrationDate > '2023-06-01' | Patients registered after June 2023 |
| Q5 | σ | σ_{Discount>15 ∧ ExpiryDate>NOW} (promotions) | WHERE Discount>15 AND ExpiryDate>GETDATE() | Active promos offering more than 15% discount |
| Q6 | σ | σ_{Duration>45 ∧ Status='Completed'} (appointments) | WHERE Duration>45 AND Status=Completed | Completed appointments lasting over 45 minutes |
| Q7 | σ | σ_{Method='Insurance' ∧ Status='Paid'} (bills) | WHERE Method='Insurance' AND Status='Paid' | Successfully paid insurance bills |

### Projection π — return only specified columns (Q8–Q10)

| Q# | Op | Formal Expression | SQL Clause | Result |
|---|---|---|---|---|
| Q8 | π | π_{FirstName,LastName,Email} (patients) | SELECT FirstName, LastName, Email | Patient contact details |
| Q9 | π | π_{FirstName,LastName,Rating} (doctors) | SELECT ... ORDER BY Rating DESC | Doctor names ranked by rating |
| Q10 | π | π_{RoomNumber,RoomType,Capacity} (rooms) | SELECT RoomNumber, RoomType, Capacity | Room registry summary |

**Natural Join ⋈ (Q11–Q15), Aggregation γ (Q16–Q20)**

| Q# | Op | Result |
|---|---|---|
| Q11 | ⋈ | Each appointment with patient and doctor full names resolved |
| Q12 | ⋈ | Appointments with department name and location |
| Q13 | ⋈ | Each doctor paired with their specialization details |
| Q14 | ⋈ | Only appointments where a promo code was applied — with code and discount |
| Q15 | ⋈ | Full billing ledger with patient name for each transaction |
| Q16 | γ | Total earnings and appointment count per doctor — ranked highest first |
| Q17 | γ | Average fee and total revenue per department for completed appointments |
| Q18 | γ | Number of appointments in each status category |
| Q19 | γ | Average doctor rating grouped by medical specialization |
| Q20 | γ | Leaderboard: top 5 highest-earning doctors |

**Set operations: Union ∪, Intersection ∩, Difference − (Q21–Q24) + Subqueries (Q25–Q30)**

| Q# | Op | Result |
|---|---|---|
| Q21 | ∪ | All people in the system — patients and doctors listed with their role |
| Q22 | ∩ | First names that appear in both the patient pool and the doctor pool |
| Q23 | − | Registered patients who have never booked a single appointment |
| Q24 | − | Registered doctors who have never completed an appointment |
| Q25 | σ+subquery | Completed appointments whose fee exceeded the overall average |
| Q26 | σ+subquery | Doctor(s) holding the highest rating in the system |
| Q27 | ⋈+γ+subquery | Patients whose total spend exceeded the average total spend per patient |
| Q28 | ⋈+subquery | The single most expensive completed appointment with the patient name |
| Q29 | ⋈+γ+HAVING | Promo codes used across more than one appointment |
| Q30 | σ+IN | Appointments that have both a confirmed Paid bill and a submitted rating |

# SECTION 3 — PHYSICAL IMPLEMENTATION

## 3.1 DDL — Database and Table Creation

| Object | Statement | Constraints Applied |
|---|---|---|
| Database | CREATE DATABASE HospitalDB | Safe reset: SET SINGLE_USER WITH ROLLBACK IMMEDIATE before DROP |
| patients | CREATE TABLE patients | PK IDENTITY \| UNIQUE(Email) \| CHECK(Email LIKE '%@%.%') \| CHECK(Gender enum) |
| doctors | CREATE TABLE doctors | PK \| UNIQUE(LicenseNumber) \| CHECK(Rating 0-5) \| CHECK(Status enum) |
| departments | CREATE TABLE departments | PK IDENTITY only |
| rooms | CREATE TABLE rooms | PK \| FK(DepartmentID) \| UNIQUE(RoomNumber) \| CHECK(RoomType) \| CHECK(Capacity 1-20) |
| promotions | CREATE TABLE promotions | PK \| UNIQUE(Code) \| CHECK(Discount 0-100) |
| appointments | CREATE TABLE appointments | 4 FK constraints \| CHECK(Status enum) \| CHECK(Fee >= 0) \| Duration NULL derived |
| bills | CREATE TABLE bills | FK(AppointmentID CASCADE) \| CHECK(Method) \| CHECK(Status) \| CHECK(Amount >= 0) |
| ratings | CREATE TABLE ratings | FK(AppointmentID CASCADE) \| UNIQUE(AppointmentID) \| CHECK(DoctorRating, PatientRating 1-5) |

**SQL — appointments table — full CREATE TABLE**

```sql
CREATE TABLE appointments (
    AppointmentID  INT           NOT NULL IDENTITY(1,1),   -- auto PK
    PatientID      INT           NOT NULL,
    DoctorID       INT           NOT NULL,
    DepartmentID   INT           NOT NULL,
    ScheduledTime  DATETIME      NOT NULL,
    EndTime        DATETIME      NULL,         -- NULL = still in progress
    Duration       INT           NULL,         -- set by trigger BR-5
    ReasonForVisit VARCHAR(300)  NOT NULL,
    Status         VARCHAR(20)   NOT NULL DEFAULT 'Scheduled',
    ConsultationFee FLOAT        NOT NULL DEFAULT 0,
    PromoID        INT           NULL,

    CONSTRAINT pk_appointments PRIMARY KEY (AppointmentID),
    CONSTRAINT fk_appt_patient FOREIGN KEY (PatientID)
        REFERENCES patients(PatientID) ON DELETE CASCADE,
    CONSTRAINT fk_appt_doctor  FOREIGN KEY (DoctorID)
        REFERENCES doctors(DoctorID),
    CONSTRAINT fk_appt_dept    FOREIGN KEY (DepartmentID)
        REFERENCES departments(DepartmentID),
    CONSTRAINT fk_appt_promo   FOREIGN KEY (PromoID)
        REFERENCES promotions(PromoID),
    CONSTRAINT chk_appt_status CHECK (Status IN
        ('Scheduled','Completed','Cancelled')),
    CONSTRAINT chk_appt_fee    CHECK (ConsultationFee >= 0)
);
```

## 3.2 DML — Data Insertion (42 rows per table)

| Table | Rows | Data Coverage |
|---|---|---|
| patients | 42 | Names, unique emails, optional phones; DOB and gender; registration dates Jan–Nov 2023 |
| doctors | 42 | Mix of Available/Busy/Off-Duty; ratings 4.4–4.9; licences LIC-D0001 to LIC-D0042; 21 specializations |
| departments | 42 | 42 unique hospital departments across 8 buildings |
| rooms | 42 | General, ICU, Operating, Recovery, Consultation; capacities 1–6; Available/Occupied/Maintenance |
| promotions | 42 | Discounts 5–50%; mix of active and expired codes for realistic test data |
| appointments | 42 | 40 Completed + 1 Scheduled + 1 Cancelled; fees $90–$350; Jan–Feb 2024 |
| bills | 42 | 40 Paid + 1 Pending + 1 Failed; distributed across Cash / Card / Insurance |
| ratings | 40 | Completed appointments only; DoctorRating 3–5; PatientRating 2–5; some NULL comments |

### SQL — Sample DML — INSERT into patients

```sql
INSERT INTO patients
    (FirstName,LastName,Email,Phone,DateOfBirth,Gender,RegistrationDate)
VALUES
('James','Anderson','james.anderson@email.com','555-2001',
 '1985-03-12','Male','2023-01-05 09:00:00'),
('Sophia','Martinez','sophia.martinez@email.com','555-2002',
 '1992-07-24','Female','2023-01-10 10:30:00'),
('Liam','Thompson','liam.thompson@email.com',NULL,
 '1978-11-05','Male','2023-01-15 11:00:00');
-- ... 39 more rows (42 total)
```

## 3.3 DQL — Select Queries (30 Queries)

The SQL script contains all 30 SELECT queries, each preceded by a PRINT label. Sample queries from each category:

### Join: appointments with department info

```
-- Q11 | appointments ⋈ patients ⋈ doctors
SELECT
    a.AppointmentID,
    p.FirstName+' '+p.LastName AS PatientName,
    d.FirstName+' '+d.LastName AS DoctorName,
    d.Specialization, a.Status, a.ConsultationFee
FROM appointments a
JOIN patients     p  ON a.PatientID = p.PatientID
JOIN doctors      d  ON a.DoctorID  = d.DoctorID;

-- Q14 | only appointments where a promo was applied
SELECT a.AppointmentID,
       p.FirstName+' '+p.LastName AS PatientName,
       pr.Code AS PromoCode, pr.Discount,
       a.ConsultationFee
FROM appointments a
JOIN patients    p  ON a.PatientID = p.PatientID
JOIN promotions  pr ON a.PromoID   = pr.PromoID;
```

### Intersection: first names in both patients and doctors

```
-- Q21 | ∪ all people in system
SELECT FirstName,LastName,'Patient' AS Role FROM patients
UNION
SELECT FirstName,LastName,'Doctor'  AS Role FROM doctors;

-- Q22 | ∩ names in both patients AND doctors
SELECT FirstName FROM patients
INTERSECT
SELECT FirstName FROM doctors;
```

### Subquery: patients who spent above the average total

```sql
-- Patients whose total spending > average spending per patient
SELECT
    p.FirstName+' '+p.LastName    AS PatientName,
    ROUND(SUM(a.ConsultationFee),2) AS TotalSpent
FROM  patients p
JOIN  appointments a ON p.PatientID=a.PatientID AND a.Status='Completed'
GROUP BY p.PatientID,p.FirstName,p.LastName
HAVING SUM(a.ConsultationFee) > (       -- HAVING not WHERE (group result)
    SELECT AVG(total)
    FROM (
        SELECT SUM(ConsultationFee) AS total
        FROM   appointments WHERE Status='Completed'
        GROUP BY PatientID
    ) sub
);
```

## 3.4 Views — 8 Reusable Query Objects

| View Name | Purpose | Tables Joined | Key Columns |
|---|---|---|---|
| vw_appointment_details | Full appointment info — all IDs resolved to names | appointments+patients+doctors+departments+promotions | PatientName, DoctorName, Department, ScheduledTime, Fee, Status, PromoCode |
| vw_doctor_summary | Per-doctor aggregated KPIs | doctors + appointments + ratings | TotalAppointments, TotalEarnings, AvgDurationMin, AvgDoctorRating |
| vw_patient_activity | Per-patient spending summary | patients + appointments + ratings | TotalAppointments, TotalSpent, AvgPatientRating |
| vw_revenue_by_department | Department-level revenue analytics | appointments + departments | Department, TotalAppointments, TotalRevenue, AvgFee |
| vw_bill_overview | Billing ledger with patient names | bills + appointments + patients | Amount, Method, BillDate, Status |
| vw_active_promotions | Non-expired promo codes only | promotions (filtered by ExpiryDate) | Code, Discount, ExpiryDate |
| vw_top_doctors | Highly rated doctors (>= 4.7) | doctors (filtered by Rating) | DoctorName, Specialization, Rating, Status |
| vw_scheduled_appointments | Active appointments awaiting completion | appointments + patients + doctors | AppointmentID, PatientName, DoctorName, ScheduledTime, Fee |

**SQL — vw_appointment_details — full definition**

```sql
CREATE VIEW vw_appointment_details AS
SELECT
    a.AppointmentID,
    p.FirstName+' '+p.LastName  AS PatientName,
    d.FirstName+' '+d.LastName  AS DoctorName,
    d.Specialization,
    dep.Name                    AS Department,
    a.ScheduledTime, a.EndTime, a.Duration AS DurationMinutes,
    a.ConsultationFee, a.Status,
    pr.Code                     AS PromoCode
FROM      appointments a
JOIN      patients     p   ON a.PatientID    = p.PatientID
JOIN      doctors      d   ON a.DoctorID     = d.DoctorID
JOIN      departments  dep ON a.DepartmentID = dep.DepartmentID
LEFT JOIN promotions   pr  ON a.PromoID      = pr.PromoID;
--        ↑ LEFT JOIN: PromoID can be NULL (no promo applied)
```

## 3.5 Indexes — 15 Performance Optimizations

| Index Name | Table | Columns | Query Pattern |
|---|---|---|---|
| idx_appt_patient | appointments | PatientID | Lookup all appointments for a specific patient |
| idx_appt_doctor | appointments | DoctorID | Doctor dashboard, earnings report queries |
| idx_appt_status | appointments | Status | Filter by Scheduled / Completed / Cancelled |
| idx_appt_time | appointments | ScheduledTime | Date-range reports and monthly revenue queries |
| idx_appt_doc_status | appointments | DoctorID, Status | Composite: doctor-specific status dispatch queries |
| idx_appt_fee | appointments | ConsultationFee | Revenue aggregation and fee distribution analysis |
| idx_appt_dept | appointments | DepartmentID | Department revenue grouping queries |
| idx_appt_promo | appointments | PromoID | Join appointments to promotions in view queries |
| idx_bills_status | bills | Status | Filter Paid / Pending / Failed bill records |
| idx_bills_appt | bills | AppointmentID | JOIN bills to appointments in view queries |
| idx_ratings_appt | ratings | AppointmentID | JOIN ratings to appointments for average computation |
| idx_rooms_dept | rooms | DepartmentID | Find all rooms in a specific department |
| idx_doctors_status | doctors | Status | Dispatch: find all Available doctors |
| idx_promo_expiry | promotions | ExpiryDate | Validate promo code expiry in real time |
| idx_patients_regdate | patients | RegistrationDate | Filter patients by registration date range |

**SQL — All 15 INDEX statements**

```sql
-- On appointments (most queried table — 8 indexes)
CREATE INDEX idx_appt_patient     ON appointments(PatientID);
CREATE INDEX idx_appt_doctor      ON appointments(DoctorID);
CREATE INDEX idx_appt_status      ON appointments(Status);
CREATE INDEX idx_appt_time        ON appointments(ScheduledTime);
CREATE INDEX idx_appt_doc_status  ON appointments(DoctorID, Status);
--                                                ↑ composite index
CREATE INDEX idx_appt_fee         ON appointments(ConsultationFee);
CREATE INDEX idx_appt_dept        ON appointments(DepartmentID);
CREATE INDEX idx_appt_promo       ON appointments(PromoID);

-- On bills, ratings, rooms, doctors, promotions, patients
CREATE INDEX idx_bills_status     ON bills(Status);
CREATE INDEX idx_bills_appt       ON bills(AppointmentID);
CREATE INDEX idx_ratings_appt     ON ratings(AppointmentID);
CREATE INDEX idx_rooms_dept       ON rooms(DepartmentID);
CREATE INDEX idx_doctors_status   ON doctors(Status);
CREATE INDEX idx_promo_expiry     ON promotions(ExpiryDate);
CREATE INDEX idx_patients_regdate ON patients(RegistrationDate);
```

## 3.6 Triggers — 7 Business Rule Automations

| Trigger | Table | Event | Business Rule |
|---|---|---|---|
| trg_calc_duration | appointments | AFTER UPDATE | Auto-sets Duration = DATEDIFF(MINUTE, ScheduledTime, EndTime) when EndTime is written — BR-5 |
| trg_update_doctor_rating | ratings | AFTER INSERT | Recalculates doctor.Rating = ROUND(AVG(DoctorRating),2) across all that doctor's appointments — BR-4 |
| trg_doctor_busy_on_appointment | appointments | AFTER INSERT | Sets doctor.Status = "Busy" when a new Scheduled appointment row is inserted — BR-6 |
| trg_doctor_available_on_complete | appointments | AFTER UPDATE | Resets doctor.Status = "Available" when appointment changes to Completed or Cancelled — BR-7 |
| trg_no_concurrent_appointments | appointments | INSTEAD OF INSERT | RAISERROR + RETURN if patient already has a Scheduled appointment — INSERT blocked entirely — BR-1 |
| trg_validate_bill_appointment | bills | INSTEAD OF INSERT | RAISERROR + RETURN if appointment.Status = "Cancelled" — INSERT blocked — BR-3 |
| trg_prevent_delete_completed | appointments | AFTER DELETE | RAISERROR + ROLLBACK TRANSACTION if any Completed appointment is in the deleted set — BR-2 |

**SQL — trg_calc_duration — auto-compute appointment duration**

```sql
CREATE TRIGGER trg_calc_duration
ON appointments AFTER UPDATE
AS BEGIN
    SET NOCOUNT ON;
    UPDATE appointments
    SET    Duration = DATEDIFF(MINUTE, i.ScheduledTime, i.EndTime)
    FROM   appointments a
    JOIN   inserted     i ON a.AppointmentID = i.AppointmentID
    WHERE  i.EndTime IS NOT NULL;
END;
```

**SQL — trg_no_concurrent_appointments — prevent duplicate active appointments (BR-1)**

```sql
CREATE TRIGGER trg_no_concurrent_appointments
ON appointments INSTEAD OF INSERT
AS BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM appointments a
        JOIN  inserted i ON a.PatientID = i.PatientID
        WHERE a.Status = 'Scheduled'
    )
    BEGIN
        RAISERROR('Patient already has an active appointment.',16,1);
        RETURN;  -- row NEVER reaches the table
    END;
    -- All clear: perform the actual INSERT
    INSERT INTO appointments
        (PatientID,DoctorID,DepartmentID,ScheduledTime,EndTime,
         Duration,ReasonForVisit,Status,ConsultationFee,PromoID)
    SELECT PatientID,DoctorID,DepartmentID,ScheduledTime,EndTime,
           Duration,ReasonForVisit,Status,ConsultationFee,PromoID
    FROM  inserted;
END;
```

## 3.7 Stored Procedures — 8 Reusable Operations

| Procedure | Parameters | Purpose |
|---|---|---|
| sp_get_patient_appointments | @PatientID INT | All appointments for a patient via vw_appointment_details — names resolved |
| sp_available_doctors | @Specialization VARCHAR(100) = NULL | Available doctors with rating — filtered by specialization if provided |
| sp_complete_appointment | @AppointmentID INT, @EndTime DATETIME, @Fee FLOAT | Marks appointment Completed with EndTime and Fee — fires BR-5, BR-7 |
| sp_apply_promo | @AppointmentID INT, @PromoID INT, @NewFee FLOAT OUTPUT | Validates promo, computes discounted fee, updates appointment.ConsultationFee |
| sp_monthly_revenue | @Year INT, @Month INT | TotalAppointments, TotalRevenue, AvgFee, MinFee, MaxFee for given month |
| sp_doctor_earnings | @DoctorID INT, @StartDate DATE, @EndDate DATE | Total completed appointments and earnings for a doctor in a date range |
| sp_register_patient | @First, @Last VARCHAR, @Email, @Phone, @DOB, @Gender | Inserts new patient and returns auto-generated IDENTITY ID |
| sp_cancel_appointment | @AppointmentID INT | Cancels a Scheduled appointment — fires BR-7; returns 0 if not found |

**SQL — sp_apply_promo — validate and apply discount code**

```sql
CREATE PROCEDURE sp_apply_promo
    @AppointmentID  INT,
    @PromoID        INT,
    @NewFee         FLOAT OUTPUT    -- returned to caller
AS BEGIN
    DECLARE @Discount FLOAT, @OldFee FLOAT;

    -- Validate promo is not expired
    SELECT @Discount = Discount FROM promotions
    WHERE  PromoID = @PromoID AND ExpiryDate > GETDATE();

    IF @Discount IS NULL
    BEGIN RAISERROR('Invalid or expired promo.',16,1); RETURN; END;

    -- Compute new fee
    SELECT @OldFee = ConsultationFee FROM appointments
    WHERE  AppointmentID = @AppointmentID;
    SET    @NewFee = ROUND(@OldFee*(1.0-@Discount/100.0), 2);

    -- Apply
    UPDATE appointments
    SET    ConsultationFee = @NewFee, PromoID = @PromoID
    WHERE  AppointmentID  = @AppointmentID;
END;
```

## 3.8 DCL — Role-Based Access Control

| Login | Role Type | Permissions Granted | Intended Use |
|---|---|---|---|
| hosp_app | Standard user | GRANT: SELECT, INSERT, UPDATE on dbo schema DENY: DELETE on dbo schema | Backend application — reads and writes data but cannot permanently delete records |
| hosp_report | Read-only | GRANT: SELECT on dbo schema only | BI dashboards, analytics tools, data exports — view access only |
| hosp_dba | DBA / Admin | ALTER ROLE db_owner ADD MEMBER — full control | Database maintenance, schema changes, index rebuild, deployment scripts |

### SQL — Full DCL script — all 3 user roles

```sql
-- Role 1: hosp_app — application login (read/write, no delete)
IF NOT EXISTS (SELECT name FROM sys.server_principals
               WHERE name='hosp_app')
    CREATE LOGIN hosp_app WITH PASSWORD='App@Secure123!';
CREATE USER hosp_app FOR LOGIN hosp_app;
GRANT SELECT,INSERT,UPDATE ON SCHEMA::dbo TO hosp_app;
DENY  DELETE             ON SCHEMA::dbo TO hosp_app;

-- Role 2: hosp_report — read-only analytics
IF NOT EXISTS (SELECT name FROM sys.server_principals
               WHERE name='hosp_report')
    CREATE LOGIN hosp_report WITH PASSWORD='Report@Secure123!';
CREATE USER hosp_report FOR LOGIN hosp_report;
GRANT SELECT ON SCHEMA::dbo TO hosp_report;

-- Role 3: hosp_dba — full database owner
IF NOT EXISTS (SELECT name FROM sys.server_principals
               WHERE name='hosp_dba')
    CREATE LOGIN hosp_dba WITH PASSWORD='DBA@Secure123!';
CREATE USER hosp_dba FOR LOGIN hosp_dba;
ALTER ROLE db_owner ADD MEMBER hosp_dba;

-- Verify permissions at any time:
SELECT dp.name AS LoginName,
       p.permission_name AS Permission,
       p.state_desc      AS State
FROM  sys.database_permissions  p
JOIN  sys.database_principals   dp
      ON p.grantee_principal_id=dp.principal_id
WHERE dp.name IN ('hosp_app','hosp_report','hosp_dba')
ORDER BY dp.name, p.permission_name;
```

## 3.9 Database Deployment

The database deploys on Microsoft SQL Server 2019 or later via a single idempotent script — safe to run multiple times without manual cleanup.

### Step-by-Step Deployment Procedure

- Install SQL Server 2019+ and SQL Server Management Studio (SSMS) if not already present.
- Open SSMS and connect to the target instance using Windows Authentication or a SQL Server login with sysadmin privileges.
- Open File > Open > SQL Script and select hospital_clinic.sql.
- The script opens with ALTER DATABASE HospitalDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE followed by DROP DATABASE. This forces all connections closed and removes any existing database.
- Execute the full script: Ctrl+Shift+Enter. The script runs top-to-bottom: DDL → DML → Indexes → Views → Triggers → Stored Procedures → DQL → DCL.
- Monitor the Messages tab. Every object creation prints a confirmation line. All 30 DQL queries execute and display results in the Results tab.
- The final block prints a complete inventory: 8 table row counts, 8 views, 7 triggers, 8 procedures, 15 indexes.
- Database is operational. Connect the application using the hosp_app login.

### SQL — Idempotent database reset — top of script

```sql
USE master; GO
IF EXISTS (SELECT name FROM sys.databases WHERE name = N'HospitalDB')
BEGIN
    ALTER DATABASE HospitalDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE HospitalDB;
END
GO
CREATE DATABASE HospitalDB; GO
USE HospitalDB; GO
```

# SECTION 4 — REPORT AND DOCUMENTATION

## 4.1 Project Conclusions

This project delivered a complete, production-quality relational database system for a hospital and clinic management platform. Every component — from conceptual analysis through physical deployment — was implemented, tested, and documented.

### What Was Achieved

- A fully normalised (1NF, 2NF, 3NF) relational schema was designed from scratch with all normalisation decisions explicitly justified.
- 8 tables with 10 foreign key relationships, 12 CHECK constraints, 6 UNIQUE constraints, and 8 DEFAULT values — all constraints enforced at the SQL Server engine level.
- 42 realistic records per table covering 21 medical specializations, multiple room types, and a realistic distribution of appointment/bill/rating statuses.
- 30 DQL queries demonstrating all 7 relational algebra operations with formal mathematical notation.
- 7 triggers enforce 7 distinct business rules — including INSTEAD OF triggers that prevent invalid state transitions before any data is written.
- 8 stored procedures encapsulate all major application operations, each documented with parameters and sample calls.
- 15 strategic non-clustered indexes on FK, status, date, and fee columns.
- 3 DCL user roles implementing least-privilege access control with GRANT, DENY, and db_owner mechanisms.

## 4.2 Key Technical Decisions and Justifications

| Decision | Justification |
|---|---|
| Surrogate IDENTITY PKs throughout all tables | Guarantees row uniqueness even when natural identifiers change (e.g. doctor renews licence). Improves FK JOIN performance. Eliminates composite PK partial dependency risk — 2NF guaranteed by design. |
| Duration stored as a derived physical column | Avoids repeated DATEDIFF computation on every reporting query. Stored as INT, auto-maintained by trg_calc_duration. Trade-off: 4 bytes per appointment row for zero per-query computation. |
| INSTEAD OF INSERT for BR-1 and BR-3 | SQL Server AFTER INSERT triggers fire after the row is already written. INSTEAD OF INSERT fires before the storage engine writes anything, allowing clean rejection via RAISERROR+RETURN without side effects. |
| AFTER DELETE + ROLLBACK for BR-2 | SQL Server prohibits INSTEAD OF DELETE on tables with cascading FK children. AFTER DELETE + ROLLBACK TRANSACTION achieves the same result — the DELETE and all cascades are rolled back atomically. |
| UNIQUE constraint on ratings.AppointmentID | Enforces one-rating-per-appointment at engine level independently of application code. Cheaper and more reliable than a trigger — the constraint is checked during INSERT at virtually zero cost. |
| ON DELETE CASCADE for patients → appointments | Supports right-to-erasure (GDPR): purging a patient account removes all their appointment history, bills, and ratings in one statement without complex multi-table DELETE chains. |
| No CASCADE on doctors → appointments | Doctor records must be preserved for financial audit trails even after a doctor leaves. Historical appointments must reference the doctor who fulfilled them — removing a doctor row would corrupt revenue reports. |
| Three distinct DCL roles (not one shared login) | Least-privilege principle: hosp_app cannot delete records; hosp_report cannot modify anything. Limits the blast radius of SQL injection attacks, accidental queries, or application bugs. |
| 42 rows of test data per table | 40 Completed + 1 Scheduled + 1 Cancelled mirrors a realistic production state. All 7 relational algebra operations are demonstrable on this dataset including set operations (UNION, INTERSECT, EXCEPT). |

## 4.3 Requirements Coverage Summary

| Sec. | Requirement | Deliverable | Status |
|---|---|---|---|
| 1.1 | Analyze problem domain and identify requirements | Functional + non-functional requirements tables | Done |
| 1.2 | Identify main data objects and business rules | 8 data object table + 12 business rules mapped to enforcement mechanism | Done |
| 1.3 | Draw use-case diagram | Framed ASCII use-case diagram: 3 actors, all use cases | Done |
| 1.4 | Draw activity diagram | Step-by-step appointment booking flow with trigger annotations | Done |
| 1.5 | Draw sequence diagram | Patient–Database–Doctor message sequence with trigger firing points | Done |
| 2.1 | Develop ER diagram | ER diagram with all 8 entities and relationships | Done |
| 2.2 | Identify entities, attributes, relationships | Per-entity attribute table for all 8 entities (Column/Type/Constraint/Description) | Done |
| 2.3 | Define relationship cardinalities and constraints | 7-relationship cardinality table with ERD notation, SQL cardinality, FK details | Done |
| 2.4 | Transform ER to relational schema | Transformation rules listed + full schema in formal notation | Done |
| 2.5 | Apply normalization 1NF, 2NF, 3NF | Each NF proven with definition, examples, and transitive dependency elimination table | Done |
| 2.6 | Relational algebra (25+ operations) | 30 queries in 7 categories with formal σ/π/⋈/γ/∪/∩/− expressions | Done |
| 3.1 | DDL — CREATE DATABASE and all tables | 1 database + 8 tables; appointments CREATE TABLE shown in full | Done |
| 3.2 | DML — 40+ records per table | 42 rows per table (40 for ratings); sample INSERT shown | Done |
| 3.3 | DQL — retrieve information using SELECT | 30 queries; 3 representative samples shown | Done |
| 3.4 | Views — store results in reusable objects | 8 views; vw_appointment_details full definition | Done |
| 3.5 | Indexes — optimize query performance | 15 indexes; all CREATE INDEX statements shown | Done |
| 3.6 | Triggers — automate operations and enforce rules | 7 triggers; 2 full definitions with comments | Done |
| 3.7 | Stored procedures — reusable logic | 8 procedures; sp_apply_promo full definition + sample calls | Done |
| 3.8 | DCL — access control | 3 user roles; full DCL script + verification approach | Done |
| 3.9 | Deploy the database system | Idempotent script; deployment steps documented; reset pattern shown | Done |

| | | | |
|---|---|---|---|
| 4.1 | Project conclusions | Section 4 — full analysis | Done |
| 4.2 | Technical decisions and justifications | 9 technical decisions with full justifications table | Done |
| 4.3 | Requirements coverage summary | This table — all requirements mapped to deliverables | Done |